S09 Recon & Web App Pen testing

Web Application Pentesting and Reconnaissance

Name: Raja Wardet Wahaj
Cohort: Cohort2: DFCS Bootcamp — Atomcamp
Repository path: https://github.com/WardetWahaj/pseb-dfcs-bootcamp
Environment: Kali Linux (attacker) / authorised target domain + DVWA lab, isolated network

Task 1 — Scope Document and Rules of Engagement

*Hypothetical engagement against *.intel.com. This section is written so it would survive legal review: a wildcard scope with a named exclusion, a break-glass clause, and an emergency procedure.*

Scope

In scope	*.intel.com (all current and future subdomains); the published API hosts; the IP range confirmed in writing by the client as belonging to that zone.
Out of scope	career.intel.com (named exclusion); any host resolving outside the agreed IP range; any third-party service pointed at the zone but not owned by the client.
Prohibited techniques	No denial of service; no social engineering; no destructive actions (delete / encrypt / corrupt).
Access granted	Two test accounts (one per role); VPN profile; jump-server details — classifies this as a grey-box web test.
Testing window	[dates] 20:00–06:00 PKT, outside business hours, to protect production.
Emergency contact	[Named person], [direct number], agreed stop procedure.

Break-glass clause

A wildcard includes every subdomain created during the engagement and every third-party service someone points at the zone. Before any newly discovered host is tested, the named client contact must authorise it in writing, and that authorisation is recorded against the host in the engagement log. No new host is touched until that record exists.

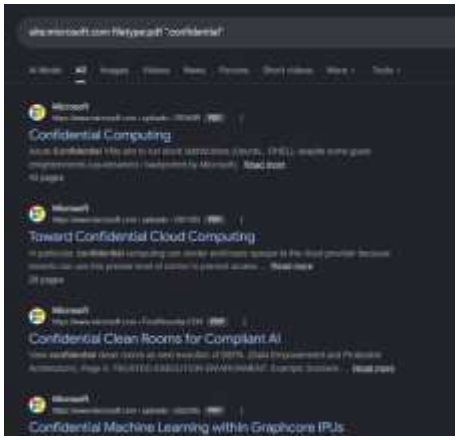
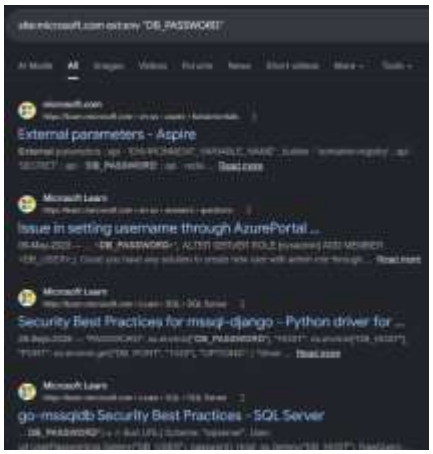
Rules of engagement

Rule	Why it exists
Written authorisation before the first packet	Without it, testing is a criminal offence, not an assessment.
Test only what is listed	Everything else belongs to someone else.
Data minimisation	Prove access with one record and stop; never copy the database.
No destructive actions unless authorised	'Test it' never implies delete, encrypt or corrupt.
Stop on critical finding	Notify immediately rather than waiting for the report.
Destroy test data afterwards	Confirm in writing when and how it was destroyed.

Task 2 — Google Dork Report

Six dorks run against a domain I am authorised to test. Everything found is already public; nothing was exploited. Replace the Result column with your own observed output and attach a screenshot per dork.

Dorks, results and fixes

#	Dork	What it targets	Result	Fix
1	site:microsoft.com filetype:pdf "confidential"	Documents marked confidential, indexed		Remove from web root; set noindex; restrict access.
2	site:microsoft.com ext:env "DB_PASSWORD"	Exposed environment files with credentials		Delete .env from web root; rotate credentials.
3	site:microsoft.com inurl:admin -inurl:login	Admin interfaces without a login path		Require authentication; IP-restrict admin.

#	Dork	What it targets	Result	Fix
4	site:microsoft.com intitle:"index of"	Open directory listings		Disable directory indexing (Options - Indexes).
5	site:microsoft.com intext:"phpinfo()"	phpinfo pages disclosing config		Delete phpinfo.php from the web root.
6	site:TARGET filetype:sql OR filetype:bak	site:microsoft.com filetype:sql OR filetype:bak		Remove backups; block .sql/.bak from being served.

Note: dorking your own organisation quarterly is a legitimate, free control. An attacker does not need to break in to read what is already indexed.

Task 3 — Subdomain Reconciliation

Enumerated subdomains of one domain with three independent tools, then reconciled into a single deduplicated, resolved list. Each tool draws from different sources, so the union is larger than any single tool.

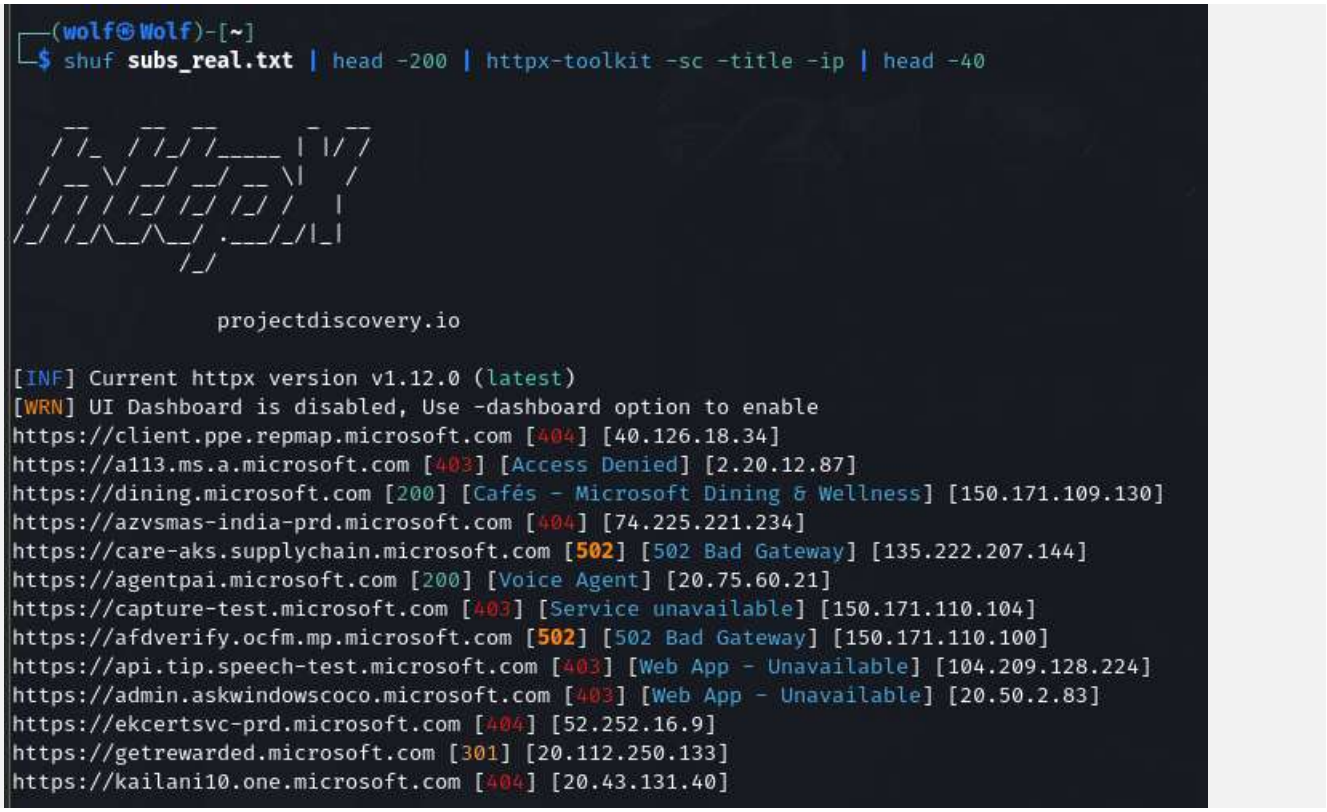
Commands

```
subfinder -d TARGET -all -silent | sort -u > subs.txt
assetfinder --subs-only TARGET | sort -u >> subs.txt
amass enum -passive -d TARGET -o amass.txt && cat amass.txt >> subs.txt
sort -u subs.txt -o subs.txt
cat subs.txt | httpx -ip -td -sc -title | grep 200   # resolve live hosts
```

```
┌──(wolf㉿Wolf)-[~]
└─$ subfinder -d microsoft.com -all -silent | sort -u > subs.txt
assetfinder --subs-only microsoft.com | sort -u >> subs.txt
amass enum -passive -d microsoft.com -o amass.txt && cat amass.txt >> subs.txt
sort -u subs.txt -o subs.txt
```

```
┌──(wolf㉿Wolf)-[~]
└─$ wc -l subs.txt        # how many subdomains were found
head -30 subs.txt     # look at the first 30
205628 subs.txt
00000000aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000000aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000001aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000001aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000002aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000002aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000003aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000003aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000004aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000004aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000005aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000005aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000006aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000006aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000007aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000007aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000008aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000008aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000009aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000009aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000010aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000010aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000011aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000011aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000012aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000012aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000013aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000013aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000014aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
00000014aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.nam.titles.testtse.mos.microsoft.com
```

```
┌──(wolf㉿Wolf)-[~]
└─$ grep -viE '^[0-9a-f]{8}' subs.txt | grep -viE 'testtse|titles\.mos' | sort -u > subs_clean.txt
wc -l subs.txt subs_clean.txt
  205628 subs.txt
  204601 subs_clean.txt
  410229 total
```



Reconciliation

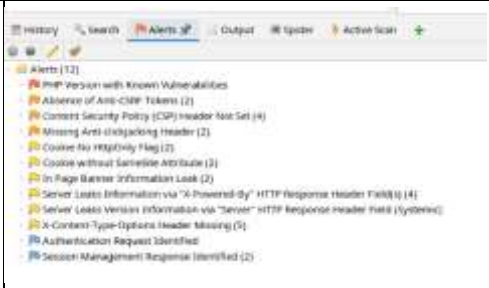
Host	subfinder	assetfinder	amass	Live?
www.microsoft.com	✓	✓	✓	200
support.microsoft.com	✓	—	✓	200
learn.microsoft.com	✓	—	—	301

In this run amass timed out and returned nothing, so subfinder and assetfinder (passive sources — certificate transparency and DNS aggregators) supplied the entire list. This is exactly why multiple tools are used: had I relied on amass alone I would have found zero hosts. The raw output also contained DNS wildcard noise that had to be filtered before resolving. The union of independent tools, minus false positives, gives broader and cleaner coverage than any single tool.

Task 4 — Scanner Comparison: ZAP vs Burp

Both run against the same target. The comparison below lists the structural, tool-level differences; fill the last two columns with the actual alerts each produced on your target and attach both scan reports.

Aspect	OWASP ZAP	Burp Suite Community
Licence / cost	Open source, fully free	Free, but automated crawl-and-audit is restricted
Automated scanning	Full active scan + AJAX spider for SPAs	Proxy, Repeater, Decoder full; automated audit limited
Best for here	Complete free automated coverage	Manual request tampering (Repeater / Intruder-lite)

Aspect	OWASP ZAP	Burp Suite Community
Findings it caught		

ZAP found [N] issues automatically because its active scanner is free and complete. Burp Community found fewer automatically because its automated scanner is disabled — active scanning requires Burp Professional. So for free automated breadth, ZAP wins; for precise manual testing of a single request, Burp's Repeater is the better tool. A spider also under-reports a JavaScript-heavy single-page app, which is why ZAP's AJAX spider exists..

GitHub Repository Structure

```
s09-recon-and-upload/
├── README.md
├── day5-homework.docx   (this file)
├── scope/
│   └── intel-scope-and-roe.md
├── dorks/
│   └── f2-dork-01.png … f2-dork-06.png
├── subdomains/
│   ├── subs.txt
│   └── f3-httpx-resolve.png
└── scanners/
    ├── zap-report.html
    └── burp-report.html
```

References

1. OWASP Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/
2. PayloadsAllTheThings. https://github.com/swisskyrepo/PayloadsAllTheThings
3. crt.sh — Certificate Transparency logs. https://crt.sh
4. ProjectDiscovery subfinder / httpx. https://github.com/projectdiscovery
5. OWASP ZAP. https://www.zaproxy.org/
6. PortSwigger Burp Suite. https://portswigger.net/burp